

03 Vector Database Comparison

Architecture, Benchmarks & Selection Guide
for Production AI Systems

DOCUMENT	3 of 6 — Knowledge Corpus
SCOPE	FAISS · Pinecone · Weaviate · pgvector · Chroma
AUDIENCE	Senior AI/ML Engineers building RAG systems
VERSION	1.0 — Assessment Corpus Release

Assessment Purpose: Part of the standardised knowledge corpus. Enables testing of **cross-document retrieval**, **multi-step reasoning** (e.g., “Which vector DB suits a cost-conscious startup needing hybrid search and ACID compliance?”), and **unanswerable question handling** (questions about Qdrant or Milvus should be gracefully declined—they are not covered here).

Contents

1	Introduction to Vector Databases	2
	Core Concepts, ANN Algorithms, SQL vs Vector DB	
2	FAISS — Facebook AI Similarity Search	2
	Index Types: Flat, IVF, HNSW, PQ; GPU support	
3	Pinecone	3
	Serverless vs. Pod-Based; Metadata Filtering; Pricing	
4	Weaviate	3
	Hybrid Search (BM25+Vector); Multi-Tenancy; Modules	
5	pgvector (PostgreSQL Extension)	3
6	Chroma	3
7	Head-to-Head Comparison Matrix	4
8	Performance Benchmarks	5
9	Selection Guide & Decision Framework	6
10	Best Practices	7

Introduction to Vector Databases

A **vector database** is a specialised data store for indexing and querying high-dimensional vectors—numerical representations (embeddings) of unstructured data. Unlike traditional databases that retrieve records by exact key lookups, vector databases find the *most semantically similar* items using approximate nearest neighbour (ANN) algorithms. They are foundational infrastructure for RAG pipelines, recommendation engines, and semantic search.

ANN Algorithms

- **HNSW:** Graph-based; best recall/speed tradeoff; high memory usage
- **IVF:** Cluster-based; probes nearest clusters; lower memory
- **PQ:** Product Quantisation; 4–16× memory reduction; some recall loss
- **LSH:** Hash-based; fast but lower recall than HNSW

Similarity Metrics

- **Cosine:** Angle between vectors — preferred for text
- **Euclidean (L2):** Straight-line distance — common for images
- **Dot Product:** Efficient unnormalised cosine

Dimension	SQL DB	Vector DB
Primary Query	Exact match, range	Semantic similarity (k-NN)
Index Type	B-tree, GIN	HNSW, IVF, PQ
Data Type	Structured rows	Float vectors + metadata
ACID	Full	Varies (often eventual)
Scale Model	Vertical + sharding	Purpose-built horizontal
Use Case	Transactional	RAG, search, recommendations

KEY INSIGHT

A 5–10% difference in recall@10 translates to a measurable drop in RAG answer faithfulness. Vector DB selection is a critical architectural decision.

SECTION 02

FAISS — Facebook AI Similarity Search

FAISS is an open-source C++ library (Meta AI, 2017) providing ANN primitives. It is a **library, not a database**—no persistence, network API, or metadata management out of the box. Offers the widest index variety with fine-grained performance control and first-class GPU support (CUDA).

Index	Recall	Speed	Memory	Best For
IndexFlatL2	100%	Slow O(n·d)	High	<100K vectors, ground truth
IndexIVFFlat	90–99%	Fast	Medium	1M–100M, moderate recall
IndexHNSWFlat	97–99.5%	Very fast (<5 ms)	High	Production, latency-sensitive
IndexIVFPQ	85–95%	Very fast	Very Low	100M+ vectors, memory-constrained

STRENGTHS

- ✓ Fastest raw performance with GPU acceleration (500K+ QPS on A100)
- ✓ Widest index variety; battle-tested at Meta scale
- ✓ MIT licence; no infrastructure cost

LIMITATIONS

- ✗ No built-in persistence, metadata filtering, or network API
- ✗ No multi-tenancy or access control
- ✗ Steep learning curve for index parameter tuning

Ideal for: Research/ML experimentation; custom vector search infrastructure; GPU-accelerated batch indexing pipelines; serving layer embedded inside a larger application.

SECTION 03

Pinecone

Pinecone (2021) is a fully managed, cloud-native vector database. All infra is abstracted behind REST/gRPC. Two modes: **Serverless** (auto-scale, per-request billing, cold starts 50–200 ms) and **Pod-based** (dedicated HNSW-in-RAM, consistent p99 <15 ms, ~\$0.096/hr per s1.x1 pod).

- **Pre-filter metadata:** Applied before ANN search. Operators: \$eq, \$ne, \$in, \$and, \$or. No recall degradation vs post-filter.
- **Namespaces:** Logical multi-tenancy within a single index.
- **Hybrid search:** Pod-based only; dense + sparse (BM25/SPLADE) via alpha parameter.
- **Compliance:** SOC 2 Type II, HIPAA-eligible. Pricing at 1M vectors / 100 QPS: serverless ~\$70–\$120/mo; pod ~\$300–\$600/mo.

STRENGTHS

- ✓ Zero infra overhead; best LangChain/LlamaIndex integration; enterprise SLAs

LIMITATIONS

- ✗ Vendor lock-in; cost scales at >50M vectors; serverless cold starts; hybrid search pod-only

SECTION 04

Weaviate

Weaviate (Go, BSD 3-Clause, 2019) is open-source with schema-based classes, GraphQL + REST + gRPC APIs, and modular vectorisation. Self-hosted (Docker/K8s) or managed WCS. Default HNSW index with configurable `ef`, `efConstruction`, `maxConnections`.

- **Hybrid search:** BM25 + vector, fused via RRF or weighted alpha. Typically improves recall@10 by **4–9%** on domain corpora vs pure vector. No external BM25 indexing required.
- **Multi-tenancy:** Native per-tenant HNSW graph isolation; hot/warm/cold tenant lifecycle for SaaS memory management.
- **Modules:** `text2vec-openai`, `text2vec-cohere`, `text2vec-huggingface`, `generative-openai`, `reranker-cohere`.

STRENGTHS

- ✓ Native hybrid search + multi-tenancy + rich module ecosystem; open-source

LIMITATIONS

- ✗ Higher ops complexity; memory-intensive (HNSW graph in RAM); WCS costly at low scale

SECTION 05

pgvector — PostgreSQL Extension

pgvector (v0.7, 2024) adds a native `vector` type and ANN operators to PostgreSQL. Operators: `<->` (L2), `<=>` (cosine), `<#>` (dot). Two index types: **ivfflat** (fast build, 85–95% recall, 10–50 ms/1M) and **hnsw** (slow build, 96–99% recall, <20 ms/1M). Available on AWS RDS, GCP Cloud SQL, Supabase, Neon.

STRENGTHS

✓ Zero new infra; full ACID; native SQL WHERE filtering; vector + tsvector in single query

LIMITATIONS

✗ Degrades beyond ~5M vectors; HNSW build locks table; lower throughput than purpose-built DBs

SECTION 06

Chroma

Chroma (Apache 2.0, 2022) is a lightweight embedding database for prototyping. Python-native; in-memory (ephemeral), persistent (SQLite + hnswlib), or client/server modes. Built-in embedding wrappers for OpenAI, Cohere, HuggingFace. Full RAG prototype in <10 lines of Python. **Not production-grade at >1M vectors**—no replication, hybrid search, or horizontal scaling. Ideal for hackathons, notebooks, CI/CD ephemeral tests, and developer onboarding.

SECTION 07

Head-to-Head Comparison Matrix

Dimension	FAISS	Pinecone	Weaviate	pgvector	Chroma
Deployment	In-process library	Fully managed SaaS	Self-hosted or WCS	PostgreSQL extension	Embedded / client-server
Scalability	Single node; GPU sharding	Auto (serverless); manual pods	Horizontal K8s	Vertical; ceiling ~5–10M	Single node; ceiling ~1M
Latency p99 (1M vectors)	<5 ms (HNSW in-mem)	5–15 ms (pod); 20–80 ms (serverless)	5–20 ms (self-hosted)	10–50 ms (HNSW)	5–30 ms (<500K)
Metadata Filter	None (app-layer)	Rich pre-filter (\$eq/\$in/\$and)	GraphQL; pre-filter	Full SQL WHERE	Basic MongoDB-style
Hybrid Search	No	Yes (pod-based)	Yes (BM25+vector, RRF)	Partial (vector + tsvector)	No
Pricing	Free (MIT)	Per RU/WU or pod-hour	Free (self-hosted)	Postgres compute only	Free (Apache 2.0)
Persistence	Manual serialisation	Managed, durable	Disk-backed, durable	Full WAL-backed	Optional (SQLite)
ACID	No	No (eventual)	No (eventual)	Yes	Partial (SQLite)
Multi-Tenancy	None	Namespaces (logical)	Native tenant isolation	PostgreSQL RLS	Collections (logical)
GPU Support	Yes (CUDA)	No (managed)	No	No	No
Production Ready	Medium (DIY ops)	High (SOC2/HIPAA)	High	High (within scale limits)	Low (prototyping)
Licence	MIT	Proprietary	BSD 3-Clause	PostgreSQL Licence	Apache 2.0

Performance Benchmarks

Benchmarks based on ANN-Benchmarks, vendor-published results, and community testing (late 2024). All 1,536-d vectors (OpenAI text-embedding-ada-002 equivalent). Single-node configurations unless noted.

8.1 Query Latency p99 (ms) by Dataset Size

System	100K	1M	10M	100M
FAISS HNSW (in-memory)	<1 ms	2–5 ms	5–12 ms	12–30 ms
FAISS IVF-PQ (in-memory)	<1 ms	1–3 ms	3–8 ms	5–15 ms
Pinecone (pod, p1.x1)	2–5 ms	5–15 ms	10–25 ms	Multi-pod
Pinecone (serverless)	10–30 ms	15–50 ms	20–80 ms	30–120 ms
Weaviate (HNSW, self-hosted)	2–5 ms	5–20 ms	15–40 ms	30–80 ms
pgvector (HNSW)	3–8 ms	10–50 ms	40–150 ms	Not recommended
Chroma (hnswlib, local)	2–10 ms	15–60 ms	Not recommended	Not recommended

8.2 Recall@10 and Throughput at 1M Vectors (16 vCPUs)

System & Index	Recall@10	p50 Latency	QPS (Single)	Index Build Time
FAISS Flat (exact)	100.0%	120–400 ms	~10	N/A
FAISS HNSW (M=32, ef=64)	98.7%	3 ms	800–1,200	~12 min
FAISS IVF-PQ (nlist=256)	91.2%	1.5 ms	3,000–6,000	~4 min
FAISS HNSW (GPU, A100)	98.5%	<0.1 ms	40,000–80,000	~45 sec
Pinecone (pod, p1)	97.5%	8 ms	~200/pod	Managed
Weaviate HNSW (ef=100)	97.8%	9 ms	400–800	~18 min
pgvector HNSW (m=16)	96.1%	22 ms	150–400	~35 min
pgvector ivfflat (lists=100)	88.5%	35 ms	200–500	~8 min
Chroma (defaults)	94.3%	18 ms	100–300	~8 min

BENCHMARK CAVEAT

Performance is highly workload-dependent. Index parameters (ef, nprobe), hardware (RAM, CPU, NVMe), embedding dimensionality, and batch vs single-query patterns all materially impact results. Always benchmark on your specific workload.

Selection Guide & Decision Framework

Scenario	Recommended	Rationale
Startup MVP / Prototype	CHROMA → pgvector	Fastest setup (<5 min); migrate to pgvector at >200K vectors or add Pinecone Serverless for managed scale
Enterprise Production RAG	PINECONE (POD)	Consistent p99 <15 ms; SOC2/HIPAA; pre-filtering metadata; minimal ops. Alternative: Weaviate K8s if hybrid search or cost at scale is a concern
Existing PostgreSQL Shop	PGVECTOR HNSW	Zero new infra; full ACID; native SQL filtering; single-DB architecture eliminates dual-write complexity. Use up to ~5M vectors
Research / GPU Experimentation	FAISS	Maximum index control; GPU acceleration (500K+ QPS on A100); add persistence + metadata layer in application code
Multi-Tenant SaaS AI	WEAVIATE	Native per-tenant HNSW isolation with hot/warm/cold lifecycle management; more robust than Pinecone namespaces at high tenant counts
Hybrid Search Required	WEAVIATE	Best native BM25 + vector fusion (RRF); 4–9% recall improvement on domain corpora vs pure vector. Pinecone pod-based as alternative

9.1 Decision Heuristics

- **Scale <1M vectors:** Any solution works; optimise for developer experience.
- **Scale 1–5M:** pgvector (if on Postgres), Pinecone Serverless, or Weaviate.
- **Scale >10M:** Pinecone pod, distributed Weaviate, or FAISS with IVF-PQ compression.
- **Latency SLA <10 ms p99:** FAISS (in-process) or Pinecone pod.
- **ACID required:** pgvector only.
- **Data residency / on-prem:** FAISS, self-hosted Weaviate, or pgvector self-hosted.
- **Budget <\$100/mo:** pgvector on existing DB or Chroma for dev.

MIGRATION PATH

Optimal progression for most RAG products: **Chroma** (prototype) → **pgvector** (early production, <2M docs) → **Pinecone or Weaviate** (scaled production). Avoid premature optimisation—the cost and complexity of purpose-built vector DBs are only justified when pgvector's performance ceiling is actually reached.

Best Practices

10.1 Index Tuning

HNSW Parameters

- **M (connections):** 16–32 typical; higher M improves recall but increases memory. Diminishing returns above M=64.
- **ef_construction:** 64–256; set $\geq 2 \times$ your target ef at query time.
- **ef (query):** Start at 50–100; tune via recall@10 on a holdout set.

IVF Parameters

- **nlist:** $\sqrt{n}$ to $4 \times \sqrt{n}$. For 1M vectors: 1,000–4,000.
- **nprobe:** Start at nlist/10; increase for higher recall.

Dimensionality

- Storage: 4 bytes $\times d \times n$. 1M vectors at 1,536d = ~6 GB before index overhead.
- OpenAI text-embedding-3-large supports dimension truncation (3,072d $\rightarrow$ 256d with ~2% recall loss).

10.2 Ingestion Strategy

- **Real-time:** Single upsert; <100 vectors/sec; use for user-facing ingestion.
- **Micro-batch (100–1K):** Async background ingestion; balances freshness and throughput.
- **Bulk (>10K):** Initial corpus load or nightly re-indexing; parallelise embedding calls (OpenAI: up to 2,048 inputs/call).
- Cache embeddings by content hash to avoid re-computing unchanged documents.

10.3 Monitoring & Observability

- **Key metrics:** query latency (p50/p95/p99), recall@k (holdout sampling), index size (GB), ingestion lag, error rate.
- **Pinecone:** `describe_index_stats()` $\rightarrow$ Prometheus custom exporter; Datadog integration available.
- **Weaviate:** Native Prometheus metrics at `/metrics`; community Grafana dashboards.
- **pgvector:** `pg_stat_user_indexes`, `pg_stat_statements`, `EXPLAIN ANALYZE`; monitor autovacuum to prevent index bloat.
- **FAISS:** No built-in monitoring—instrument at application layer; periodically validate recall vs flat index ground truth.

10.4 Migration Strategy

1. **Export** embeddings + metadata to Parquet (portable format).
2. **Validate** embedding model compatibility—changing models requires full re-embedding.
3. **Dual-write** to both systems during transition to prevent gaps.
4. **Shadow-test** retrieval quality by comparing both systems on the same query set.
5. **Blue/green cutover:** Shift read traffic 10% $\rightarrow$ 50% $\rightarrow$ 100% while monitoring p99 and recall.

10.5 Common Pitfalls

- Not normalising embeddings before cosine search (causes incorrect rankings).
- Over-indexing small collections—HNSW overhead not justified for <10K vectors.
- Ignoring query-time ef tuning—default ef settings often sacrifice recall for latency unnecessarily.
- Storing raw embeddings without metadata—makes post-retrieval filtering impossible.
- Committing secrets or API keys in vector ingestion scripts.